

Java HotSpot VM Cheatsheet

L0 Essence: The essence of the HotSpot VM is to execute Java code using high-speed assembly Templates for interpreter decoding, Tiered C1/C2 JIT compilation, and speculative compilation/deoptimization (Bailout) mechanisms, while utilizing JMM logical memory barriers to enforce ordering and visibility on weak hardware memory models, combined with Loom virtual thread Mount/Unmount stack freezing and ZGC load barrier pointer self-healing to achieve near-zero GC pauses.

[M1.1] HotSpot Class Loading Phases & Parent-Delegation Underlying Implementation

The class loading process consists of Loading (obtaining class binary stream), Linking (Verification of bytecode format, Preparation of static fields with default values, Resolution of symbolic references to direct pointers), and Initialization (running `<clinit>` static constructors). Parent-delegation is implemented in `ClassLoader.loadClass()`: it first searches the loaded class cache, and if not found, recursively delegates class loading to the parent loader. The local `findClass()` is only executed if the parent loader fails (throws `ClassNotFoundException`), which fundamentally secures the singleton nature and safety of core `java.lang.*` libraries.

[M1.2] Runtime Metaspace & Klass Metadata Layout

From JDK 8 onwards, the Permanent Generation (PermGen) was deprecated and replaced by Metaspace, which utilizes Native Memory. Every Java class has a corresponding C++ structure `Klass` that describes its vtable, field offsets, and inheritance hierarchy. The header of a Java object in the heap consists of the Mark Word and the Klass Word, where the Klass Word stores a direct pointer to the `Klass` structure in Metaspace for rapid runtime type checks.

[M1.3] Bytecode Interpreter Implementation: Template Interpreter Speedup Principles

A standard bytecode interpreter relies on a giant `switch(bytecode)` loop, causing high branch misprediction rates and function call overhead during dispatch. HotSpot adopts the Template Interpreter: during JVM initialization, a template generator runs through all bytecodes, translating each into a small segment of raw assembly instructions (Templates). Operands are passed directly via dedicated CPU registers (such as `r13` as the bytecode pointer in x86), using direct threaded code jumps to achieve peak execution efficiency.

[M1.4] Stack Frame Physical Layout & Operand Stack/Local Variable Table Interaction

HotSpot builds a stack frame on the physical system stack when invoking Java methods. The frame header stores dynamic links, method return addresses, and the method's active `Klass` pointer. The Local Variable Table holds parameters and local variables in indexable slots, while the Operand Stack acts as a temporary workspace for evaluations. Instructions load data from the local variable table (e.g., `iload`) onto the operand stack, perform operations (e.g., `iadd`), and store results back (e.g., `istore`) to the local variables, driving fast stack-to-register exchange.

[M2.1] JMM Epistemology: Ordering, Visibility, Atomicity & Happens-Before

Since modern multi-core CPUs use independent Store Buffers and caches, the hardware represents a weak memory consistency model. The Java Memory Model (JMM) is a software-level specification that abstractly guarantees Visibility, Ordering, and Atomicity for multi-threaded programming. Happens-Before is the core epistemology of JMM: it defines a partial order (e.g., a volatile write happens-before subsequent reads, a lock release happens-before subsequent lock acquisitions). If a Happens-Before relation exists between two operations, JVM ensures visibility and execution order.

[M2.2] Instruction Reordering & CPU Out-of-Order Execution Physics

To maximize CPU pipeline utilization and minimize stalls, the JIT compiler reorders bytecodes (e.g., grouping local variable writes) without violating single-thread execution semantics. In multi-core hardware, CPU cores employ Out-of-Order execution to run ready instructions early and write to Store Buffers asynchronously. However, this asynchronous buffering delays cache synchronization across cores, potentially leading to read-write data inconsistencies and visibility issues in concurrent threads.

[M2.3] Hardware-Level Memory Barriers (Fences) Design

To provide strong synchronization semantics, multi-core CPUs include hardware memory barrier instructions. In x86, barriers comprise `lfence` (prevents read-read/read-write reordering), `sfence` (prevents write-write reordering), and `mfence` (a full barrier that flushes the Store Buffer and invalidates peer cache lines). In ARM, `dmb` and `dab` are used. These barriers block instruction reordering across their boundary, forcing local Store Buffer flushes to maintain global memory order.

[M2.4] Four JVM Barrier Types (LoadLoad, LoadStore, StoreStore, StoreLoad) & Volatile

The JVM spec abstracts hardware fences into four logical barriers: `StoreStore`: Guarantees that preceding writes are visible before subsequent writes. `LoadLoad`: Guarantees that preceding reads are completed before subsequent reads. `LoadStore`: Guarantees that preceding reads are completed before subsequent writes. `StoreLoad`: Forces preceding writes to be visible before subsequent reads. A volatile write is preceded by a `StoreStore` and followed by a `StoreLoad` (compiled as a `lock addl` or equivalent instruction on x86 to flush the Store Buffer). Volatile reads are followed by `LoadLoad` and `LoadStore` barriers to prevent subsequent reordering and ensure multi-core visibility.

[M3.1] Tiered Compilation & C1/C2 Compiler Cooperation

To balance application startup latency and peak execution performance, HotSpot uses a Tiered Compilation model: `Tier 0`: Interpreted execution (Template Interpreter). `Tier 1-3`: C1 Compiler (Client Compiler) tiers, performing fast JIT compilation and gathering runtime Profiling data (e.g., branch probabilities, class type feedback). `Tier 4`: C2 Compiler (Server Compiler) tier, consuming C1 profiles to execute global optimization passes (e.g., loop unrolling, devirtualization) and generate highly optimized native machine code.

[M3.2] Compilation Thresholds & JIT Activation Calculations

JVM monitors code activity using two method-level counters: the Invocation Counter and the Backedge Counter (tracking loops). Under tiered compilation, JIT compilation triggers dynamically based on compilation queue load: $\text{when InvocationCounter} + \text{BackedgeCounter} > \text{Threshold} \times S$ (where S is a scaling factor based on compilation queue length), the thread submits an asynchronous compilation request to the compile Broker to upgrade execution to C1 or C2.

[M3.3] Escape Analysis & Scalar Replacement

During C2 optimization passes, Escape Analysis traces the pointer propagation paths of newly allocated objects. If it determines that an object does not escape the current method boundary (i.e., it is not returned or assigned to global variables), the compiler performs Scalar Replacement: it skips heap allocation and GC tracking entirely, instead breaking the object's fields down into independent scalar values allocated on the stack frame or mapped directly to registers.

[M3.4] Speculative Optimization & Deoptimization (Bailout)

JIT compilers utilize speculative optimization (e.g., assuming a virtual method has a single class implementation and inlining it). If this assumption is violated at runtime (e.g., a new implementation class is dynamically loaded), execution branches to a precompiled Trap node, triggering Deoptimization: the JVM pauses execution, extracts register values, reconstructs the corresponding interpreter stack frame, and transfers execution back to the interpreter.

L1 Logical Framework

`Template Execution & Metaspace`: Class metadata `Klass` resides in the native heap Metaspace, while the Template Interpreter generates dynamic assembly sequences to bypass switch-loop decode dispatch and pass stack frame operands directly. `JMM Visibility & Barrier Ordering`: JMM guarantees thread synchronization via Happens-Before, placing hardware-level memory barriers before/after volatile read-write accesses to flush Store Buffers and prevent CPU instruction reordering. `Tiered Compilation & Speculative Deopt`: JIT compilation divides work between fast C1 profiling and heavy C2 global optimization, implementing deoptimization to reconstruct stack frames and slide execution back to the interpreter if speculative compiler invariants fail. `Self-Healing Barriers & Virtual Threads`: ZGC utilizes colored pointers and load barriers to resolve stale pointers in parallel relocation sets via self-healing, while Project Loom coordinates user-space virtual threads by freezing/thawing stack frames on carrier threads.

L2 Data Flow Topology

`Java Bytecode` `Template Interpreter` `Method Counters`
`Incremented` `Triggers C1/C2 JIT` `Speculative Optimization` `Compiled`
`Machine Code` `Volatile Write (StoreStore + StoreLoad)` `Forces`
`Store Buffer Flush` `Hardware Cache Coherence` `Type Guard Fails`
`Deoptimization Trap` `Reconstruct interpreter Frame` `Bailout to`
`Interpreter` `Object Allocation` `ZGC Relocate Set` `Concurrent Copy`
`Write to Forwarding Table` `Thread reads Old Pointer (Marked)` `Load`
`Barrier Intercept` `Lookup Forwarding Table` `Self-heal Pointer to`
`Remapped` `Return active reference.`

Trade-off Matrix: Lock & Compilation

[M4.1] Object Header Mark Word Layout & Lock State Bits

Every Java object header contains a Mark Word whose bit width matches the CPU architecture (32/64 bits). It reuses its bits depending on lock states: in the unlocked state, it stores the Hash-Code, age bits, and lock tag 01; in the biased lock state, it holds the biased Thread ID, bias bit 1, and lock tag 01; in the lightweight lock state, the Mark Word points to a BasicObjectLock record on the thread's stack, with lock tag 00; in the heavyweight lock state, it points to a C++ ObjectMonitor structure, with lock tag 10.

[M4.2] Biased Locking Acquisition, Revocation & Deprecation

Biased locking assumes that a lock is usually acquired by the same thread repeatedly. When first acquired, the thread ID is written to the Mark Word via a CAS. Subsequent re-entries incur near-zero overhead. However, if another thread tries to acquire the lock, bias revocation requires a global Safepoint to pause the holding thread and upgrade the lock to lightweight. In concurrent, multi-threaded microservice environments, Safepoint pauses outweigh the synchronized instruction savings, leading to biased locking being deprecated and disabled by default since JDK 15.

[M4.3] Lightweight Locking & CAS Stack Lock Records (BasicObjectLock)

If biased locking is disabled or a lock encounters mild competition, it upgrades to a lightweight lock. The thread allocates a BasicObjectLock (Lock Record) on its stack frame. The JVM uses a CAS to write the address of this Lock Record into the object's Mark Word. If successful, the thread has acquired the lightweight lock. If the CAS fails, it indicates multi-threaded competition, preparing the lock for heavyweight inflation.

[M4.4] Heavyweight Locking & ObjectMonitor Inflation Model

If lightweight CAS attempts fail, the lock inflates to heavyweight. JVM calls ObjectSynchronizer::inflate() to allocate a C++ ObjectMonitor and point the Mark Word to it. The Monitor maintains an _EntryList containing threads blocked waiting for the lock, and a _WaitSet holding threads suspended by wait(), ultimately employing OS mutexes to manage thread blocking and wake-up.

[M5.1] ZGC 64-bit Virtual Address Space Colored Pointers Layout

ZGC stores GC metadata directly in object references rather than the object header. The 64-bit colored pointer layout contains the active heap address in its low 42 bits (enabling a maximum 16TB heap size) and maps 4 bits for color status: Marked0, Marked1 (tracking object liveness across cycles), Remapped (confirming the pointer has been updated to the relocated address), and Finalizable.

[M5.2] ZGC Concurrent Relocation & Forwarding Tables

During the Relocation phase, ZGC copies surviving objects from the Relocation Set (pages to be reclaimed) to new pages, removing fragmentation. Since this happens concurrently with application threads, ZGC maintains a Forwarding Table in the old page to map old addresses to new addresses, using CAS to resolve race conditions between GC and application threads.

[M5.3] ZGC Load Barrier & Self-Healing Pointer Remapping

When a Java thread reads an object reference from a field, the ZGC Load Barrier is executed:
 \textbf{Fast Path}: Checks pointer color bits. If the reference is already Remapped, execution continues directly (taking just a few instructions).
 \textbf{Slow Path}: If color bits match Marked but not Remapped, the barrier intercepts the read, queries the page's Forwarding Table, retrieves the new address, and updates the active field reference (referred to as \textbf{Self-Healing}). It then updates the color bits to Remapped so subsequent lookups hit the Fast Path.

[M5.4] ZGC Memory Page Divisions & Fine-Grained Page Management

ZGC skips fixed generational boundaries and partitions the heap into three dynamically sized Page types:
 \textbf{Small Page (2MB)}: Allocates small objects up to 256KB.
 \textbf{Medium Page (32MB)}: Allocates medium-sized objects between 256KB and 4MB.
 \textbf{Large Page (Variable size, must be a multiple of 2MB)}: Each page contains exactly one object larger than 4MB. Large Pages do not undergo relocation to avoid the massive bandwidth cost of copying large memory structures.

[M6.1] G1 Region Divisions & Remembered Set (RSet) Card Tables

The G1 collector divides the heap into thousands of equal-sized Regions. To allow independent Region collections (Mixed GC) without scanning the entire heap, each Region maintains a Remembered Set (RSet) recording incoming references from other Regions. Write barriers intercept reference mutations and record them in the RSet, allowing the GC to load RSets as root sets directly.

[M6.2] Shenandoah GC: Brooks Pointers & Concurrent CAS Redirection

Shenandoah is a low-latency GC that uses Brooks Pointers for concurrent relocation. It prepends a Brooks Pointer slot to each object header, pointing to the object itself under normal conditions. When relocating an object, the GC copies it and uses a CAS to redirect the old object's Brooks Pointer to the new copy. Application threads reading or writing to the old reference are automatically redirected to the new copy via the Brooks Pointer.

[M6.3] Project Loom Virtual Threads Carrier Architecture

Standard Java platform threads map 1:1 to OS kernel threads, making thread creation, destruction, and context switching expensive due to kernel-user space transitions. Project Loom introduces user-space Virtual Threads. Millions of virtual threads can run concurrently by multiplexing onto a pool of platform carrier threads (e.g., ForkJoinPool), scheduled in user space to bypass OS context switch costs.

[M6.4] Virtual Thread State Machine Transitions: Mount/Unmount Stack Freeze/Thaw

When a virtual thread executes a blocking JVM I/O call (e.g., Socket.read()) or calls LockSupport.park(), the JVM intercepts the call and performs an \textbf{Unmount}: it copies the virtual thread's stack frames to the Java heap. When the I/O event registers as ready, the scheduler performs a \textbf{Mount}: it copies the stack frames back to an available carrier thread to resume execution.

[M6.5] Virtual Thread Pinning Scenarios & ReentrantLock Alternatives

If a virtual thread attempts blocking operations inside a synchronized block or method, or while executing native code (JNI), the thread becomes pinned to its carrier thread. This prevents the JVM from executing an Unmount, causing the underlying platform carrier thread and OS thread to block. To avoid carrier starvation under high load, developers should replace synchronized with ReentrantLock.

[M6.6] Concurrent Marking SATB Barriers & Tri-Color Synchronization

To prevent application threads from overriding references and causing GC threads to skip active objects, G1 uses SATB (Snapshot-At-The-Beginning) write barriers. When a reference deletion is intercepted, the barrier colors the overridden reference gray and saves it to a local SATB buffer. The GC treats these references as live relative to the initial snapshot, preserving marking consistency.

[M6.7] GC Card Tables & Dirty Card Tagging

HotSpot divides old space memory into 512-byte Cards. An associated Card Table byte array maps the state of each card. When a write barrier intercepts a reference update that points from old space to new space, it runs card_table[addr >> 9] = 0 to mark the card as Dirty. Minor GCs only scan Dirty Cards, avoiding old-generation traversals.

[M6.8] HotSpot Safepoints & Safe Regions

To perform global tasks like GC or lock revocation, the JVM must suspend all application threads at a Safepoint. Threads poll for Safepoint flags at loop boundaries, method returns, and invocation exits using safepoint instructions (e.g., reading a dedicated safepoint page that the JVM marks as read-only to trigger a hardware segment fault, suspending the thread). Sleeping or blocked threads are considered in a Safe Region; the GC can run concurrently without waiting for them to wake up.

Zone T1: HotSpot parameters & flags

-XX:+UnlockDiagnosticVMOptions -XX:PrintAssembly : Prints native assembly instructions compiled by C1/C2 JIT (requires the hsdis dynamic library).
 \-XX:+UseZGC / -XX:+UseGenerationalZGC : Enables the low-latency ZGC collector, or the generational ZGC model.
 \-XX:+TraceDeoptimization : Traces and logs JIT deoptimization reasons, target signatures, and stack frame reconstructions.
 \-XX:CompileThreshold=10000 : Sets the invocation counter threshold to trigger C2 JIT compilation.
 \-XX:-UseBiasedLocking : Disables biased locking to bypass revocation latency, upgrading directly to lightweight locking.

Zone T2: Fences & Lock diagnostics

lock addl \$0x0, (%rsp) / lock orl \$0x0, (%rsp) : Assembly output generated after a volatile write on x86. Acts as a StoreLoad barrier by forcing a Store Buffer flush and invalidating peer cache lines.
 \mov 0x8(%rdi), %r10 ; cmp %r10, MAP_SMI ; jne deopt_stub : JIT Map check pattern. Extracts the class pointer, compares it with a compiled Klass address, and branches to a deopt stub on mismatch.
 \0x0000000000000005 (MarkWord ending = 105) : Indicates the object is in a heavyweight locked state, where the MarkWord stores the address of the corresponding ObjectMonitor.
 \test %eax, 0x160000(%rip) (Safepoint Poll) : Safepoint poll check instruction placed at loop exits and method returns. When the page is marked read-only, reading triggers a SIGSEGV signal to suspend the thread.